Gestion des Exceptions & Curseurs en MySQL

Contexte réel

Imagine que tu développes une application pour une pharmacie :

- Le caissier ajoute un client
- Le système crée une commande
- Le stock doit être mis à jour
- Le ticket de caisse doit être validé

Problèmes possibles :

- On oublie le nom du client (champ NOT NULL)
- Un produit n'existe pas
- Deux agents ajoutent le même email
- Une requête retourne zéro résultat ou plusieurs résultats

Sans gestion d'exception : l'application plante

Solution MySQL :

- **HANDLERS** → capturer et gérer les erreurs pour que le programme continue ou sorte proprement
- **CURSORS** → lire les résultats d'un SELECT ligne par ligne, comme dans une boucle "for"

Gestion des Exceptions (HANDLER)

Définition :

- Une **exception** = une erreur qui survient pendant l'exécution d'un programme SQL.
-

Formateur : AARAB HAFID

- Un **handler** = un plan B qui indique à MySQL quoi faire quand une erreur apparaît, plutôt que de laisser le programme planter.

Exemples d'erreurs fréquentes :

- 1048 → champ obligatoire (NOT NULL) manquant
- 1062 → doublon de clé primaire
- 1146 → table inexistante
- NOT FOUND → SELECT INTO ne renvoie aucune ligne

Syntaxe :

Requête: **DECLARE** handler_action **HANDLER FOR** condition_value statement;

Décomposition pour débutants :

1. **handler_action** → ce que MySQL doit faire si une erreur survient
   - CONTINUE → continuer le programme malgré l'erreur
   - EXIT → sortir du bloc courant (comme un "break" dans une boucle)
2. **condition_value** → quelle erreur doit déclencher le handler
   - SQLSTATE → code d'erreur standard SQL
   - Code MySQL spécifique (ex : 1048, 1062)
   - SQLWARNING → avertissements SQL
   - NOT FOUND → aucune ligne trouvée (SELECT INTO ou curseur)
   - SQLEXCEPTION → toutes les erreurs restantes
3. **statement** → instructions à exécuter quand l'erreur apparaît
   - **Exemple :** afficher un message, mettre une variable à 1, ou quitter le bloc

En résumé :

- Tu déclares un **handler** pour une erreur spécifique

- Tu dis à MySQL : “Si cette erreur arrive, fais ça plutôt que de planter”

Exemple HANDLER :

Contexte réel :

<< **Tu veux chercher un client par son nom. S’il n’existe pas, tu ne veux pas que ton programme plante.** >>

Requête:

```
DELIMITER //
CREATE PROCEDURE simple_handler_example()
BEGIN
  -- 1. On crée une variable pour savoir si une erreur survient
  DECLARE erreur_occur BOOLEAN DEFAULT 0;
  -- 2. On déclare un HANDLER EXIT pour l'erreur NOT FOUND
  -- Cela signifie : si aucune ligne n’est trouvée, MySQL exécute le handler et sort du bloc
  DECLARE EXIT HANDLER FOR NOT FOUND
    SET erreur_occur := 1;
  -- 3. Tentative de récupérer un client
  -- SELECT INTO permet de mettre le résultat dans une variable
  SELECT nom INTO @nom_client
  FROM clients
  WHERE id = 999; -- supposons que ce client n’existe pas
  -- 4. Si aucune erreur → affichage du résultat
  IF erreur_occur = 0 THEN
    SELECT CONCAT('Client trouvé : ', @nom_client) AS message;
  END IF;
```

```
-- 5. Si l'erreur a été détectée → message clair
  IF erreur_occur = 1 THEN
    SELECT 'Client non trouvé' AS message;
  END IF;
END //
DELIMITER ;
```

Explications :

1. DECLARE erreur_occur BOOLEAN DEFAULT 0; → on prépare un "drapeau" pour savoir si l'erreur survient.
2. DECLARE EXIT HANDLER FOR NOT FOUND SET erreur_occur := 1; → si SELECT INTO ne trouve rien, on met la variable à 1 et sort du bloc.
3. SELECT nom INTO @nom_client ... → tente de récupérer le client.
4. IF erreur_occur = 0 THEN ... → si tout va bien, affiche le client.
5. IF erreur_occur = 1 THEN ... → si erreur, affiche un message clair pour l'utilisateur.

Curseurs (CURSOR)

Définition :

- Un **curseur** permet de parcourir les résultats d'un SELECT **ligne par ligne**.
- C'est comme une boucle for dans les langages classiques.
- **Propriétés :** lecture seule, non navigable, utilise une copie temporaire des données.

Contexte réel :

Tu veux envoyer un email à tous tes clients, ou mettre à jour une information ligne par ligne. Le curseur te permet de parcourir chaque client individuellement.

Syntaxe et étapes :

1. **Déclarer le curseur**

Requête: **DECLARE** nom_curseur CURSOR FOR SELECT ...;

2. **Ouvrir le curseur**

Requête: **OPEN** nom_curseur;

3. **Récupérer la ligne suivante**

Requête: **FETCH** nom_curseur INTO ...;

4. **Fermer le curseur**

CLOSE nom_curseur;

Important : toujours déclarer un **CONTINUE HANDLER FOR NOT FOUND** pour savoir quand il n'y a plus de lignes.

Exemple CURSOR :

Contexte réel : << parcourir tous les clients et construire une liste. >>

Requête:

```
DELIMITER //
-- INOUT On envoie une valeur → la procédure la modifie → et renvoie la nouvelle valeur
CREATE PROCEDURE simple_cursor_example(INOUT txt VARCHAR(1000))
BEGIN
  -- 1. Variable pour savoir si on a fini de parcourir toutes les lignes
  DECLARE finished INT DEFAULT 0;
  -- 2. Variable pour stocker le nom du client récupéré
  DECLARE v_nom VARCHAR(100);
  -- 3. Déclarer le curseur sur les noms des clients
  DECLARE cur_client CURSOR FOR
```

```
    SELECT nom FROM clients;

  -- 4. Handler pour savoir quand le curseur a fini
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET finished = 1;
  -- 5. Ouvrir le curseur
  OPEN cur_client;
  -- 6. Boucle pour parcourir toutes les lignes
  boucle_clients: LOOP
    -- Récupère la ligne suivante
    FETCH cur_client INTO v_nom;
    -- Si plus de résultats → sortir de la boucle
    IF finished = 1 THEN
      LEAVE boucle_clients;
    END IF;
    -- Ajouter le nom à la chaîne texte
    SET txt = CONCAT(txt, '\nClient : ', v_nom);
  END LOOP;
  -- 7. Fermer le curseur
  CLOSE cur_client;
END //
DELIMITER ;
```

Explications :

1. DECLARE finished INT DEFAULT 0; → savoir si le curseur a atteint la fin.
2. DECLARE cur_client CURSOR FOR ... → prépare le SELECT pour le curseur.
3. DECLARE CONTINUE HANDLER FOR NOT FOUND SET finished = 1; → lorsqu'il n'y a plus de lignes, on marque "fini".

4. OPEN cur_client; → initialise le curseur.
5. FETCH cur_client INTO v_nom; → récupère chaque ligne.
6. IF finished = 1 THEN LEAVE boucle_clients; → sortir de la boucle quand terminé.
7. CLOSE cur_client; → libère les ressources.